สไลด์สรุปสำหรับคนพรีเซนต์

LabHub
ระบบจัดการห้องแล็บ IoT

28 หน้า อ่านจบแล้วเล่าได้ว่าแอปทำอะไร มีอะไร และสร้างมายังไง
ไม่ต้องรู้เรื่องโค้ด · อยากรู้ลึกกว่านี้ เปิด**ฉบับเต็ม**ได้เลย

ภาพหน้าจอใช้ข้อมูลสมมติ (ชื่อคนกับรหัสนักศึกษาไม่ใช่ของจริง) · กล่องส้ม = ตัวอย่างคำพูด ไม่ต้องท่อง

LabHub คืออะไร

พูดง่าย ๆ คือ แอปดูแลห้องแล็บ IoT

ก่อนมีแอป แก้ให้ตรงกับที่เจอจริง

- จดยืมในสมุด หายง่าย ไม่รู้ว่าของอยู่กับใคร
- ไม่รู้ว่าของว่างไหม ต้องเดินไปถาม
- ไม่มีใครเตือนตอนครบกำหนดคืน
- คอมเสีย ไม่มีช่องทางแจ้ง

ยืม-คืนอุปกรณ์ IoT

สแกน QR ถ่ายรูป ส่งคำขอ ผู้ดูแลกดอนุมัติในแอป

ดูแลห้องคอม

เห็นเครื่องว่าง/เสีย แตะแจ้งปัญหาได้ทันที

ลองพูดแบบนี้

LabHub ทำให้การยืม-คืนของและการดูแลห้องคอม **มีหลักฐาน ตรวจสอบได้ และเลิกใช้กระดาษ** ใช้ได้ทั้งบนมือถือและเว็บ

ใครใช้บ้าง

3 บทบาท เข้าด้วย Google มข.

นักศึกษา

ดูของว่าง · ขอยืม/คืน/ยืมต่อ · นัดรับ · ดูประวัติตัวเอง · ดูผังห้องคอม · แจ้งปัญหา

TA (ผู้ช่วย)

อนุมัติ/ปฏิเสธคำขอ · ตรวจของตอนคืน · นัดเวลารับของ · งานซ่อม · ตรวจประจำเทอม

Admin

ทุกอย่างของ TA + เพิ่ม/แก้อุปกรณ์ · ตั้งกติกา · แต่งตั้ง TA · เพิ่มห้อง/เครื่อง

เข้าครั้งแรกระบบสร้างบัญชีให้เอง ไม่ต้องสมัคร · **TA อนุมัติคำขอของตัวเองไม่ได้** · ต้องมี Admin เหลืออย่างน้อย 1 คนเสมอ

ยืม-คืน

ยืมของ 1 ครั้ง ผ่านอะไรบ้าง



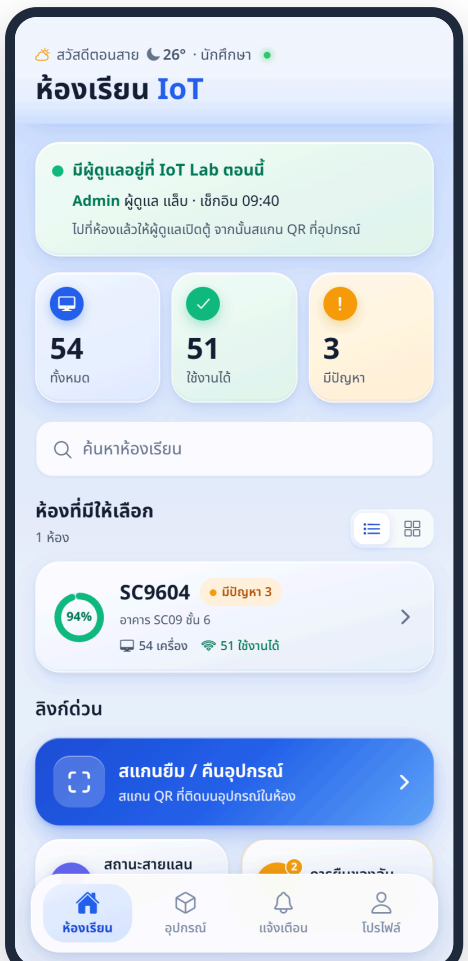
3
ชิ้นสูงสุดต่อคน (นับคำขอที่รอด้วย)

3/5/7
วัน ให้เลือกระยะยืม

30 นาที
ไม่มีใครตอบ คำขอยืมหมดอายุ

1 ครั้ง
ยืมต่อได้ (ก่อนเลยกำหนด)

คืนก็แบบเดียวกัน: สแกน → ขอคืน → ถ่ายรูป → ผู้ดูแลตรวจรับ (ไม่มีใครตอบใน 30 นาที ระบบบันทึกคืนให้เอง)



เปิดแอปมา เห็นอะไร

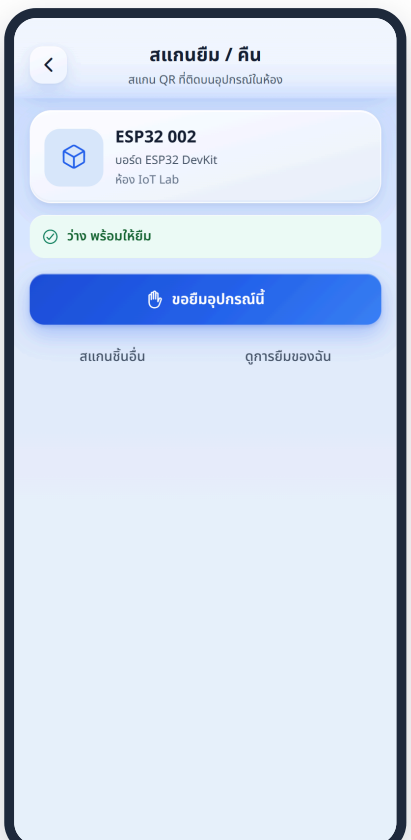
- **แถบเขียว** ตอนนี้มีผู้ดูแลอยู่ห้องไหม
- **ตัวเลข 3 ช่อง** คอมทั้งหมด / ใช้ได้ / มีปัญหา
- **การ์ดห้อง** กดเข้าไปดูผังห้องคอม
- **ปุ่มฟ้า** ทางลัดไปสแกนยืม-คืน

💬 **ลองพูดแบบนี้**

เปิดแอปมาก็รู้เลยว่ามีผู้ดูแลอยู่ห้องไหม ไม่ต้องเดินไปเก้อ



ของว่างกี่ชิ้น



สแกนแล้วขอยืม

เช็กก่อน แล้วสแกนยืม

- รวมเป็นรุ่น เช่น NodeMCU **ว่าง 5 จาก 8** ของหมดก็บอกว่าน่าจะว่างวันไหน
- สแกน QR ที่ตัวของ รู้เลยว่าว่างไหม
- เลือกวัน บอกสภาพ **ถ่ายรูปสด** ยอมรับกฎ แล้วส่ง
- สแกนไม่ติด? พิมพ์**รหัส 4 ตัว**ใต้ QR แทน

ทำไมต้องถ่ายรูป? เป็นหลักฐานสภาพของ ถ้าคืนมาเสีย ผู้ดูแลเทียบรูปก่อน-หลังได้ เลยต้องยืมบนมือถือ



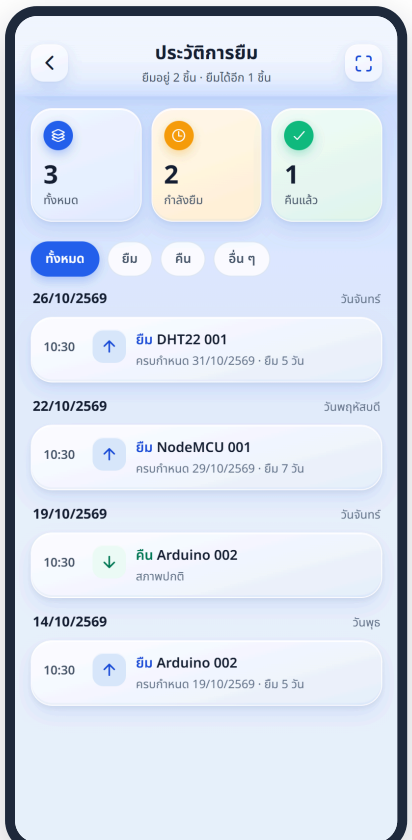
ไม่มีผู้ดูแลอยู่ห้อง? นัดรับ

- กด **"ขอยืมแบบนัดรับ"** ฝากคำขอไว้ก่อน
- ผู้ดูแลเลือกเวลานัด แอปแจ้งกลับ
- มาตามนัด สแกน → **"ยืนยันรับของตามนัด"**
- แอป**กันของไว้ให้** คนอื่นแย่งไม่ได้

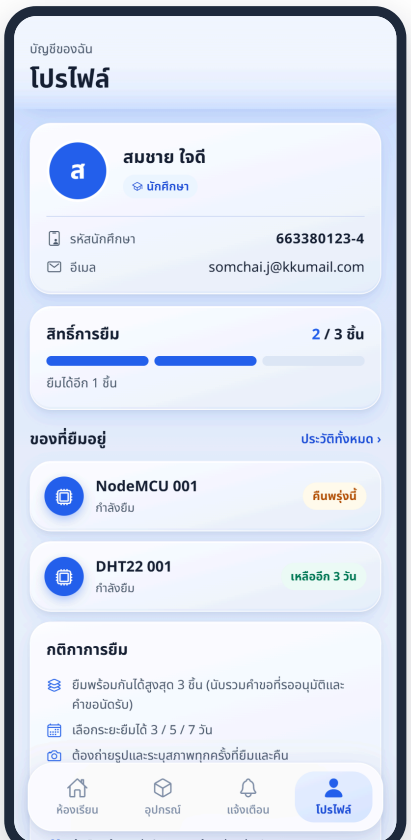
💬 **ลองพูดแบบนี้**

ฟีเจอร์นี้อาจารย์ที่ปรึกษาแนะนำมา แก้ปัญหานักศึกษาไปแล้วไม่เจอผู้ดูแล

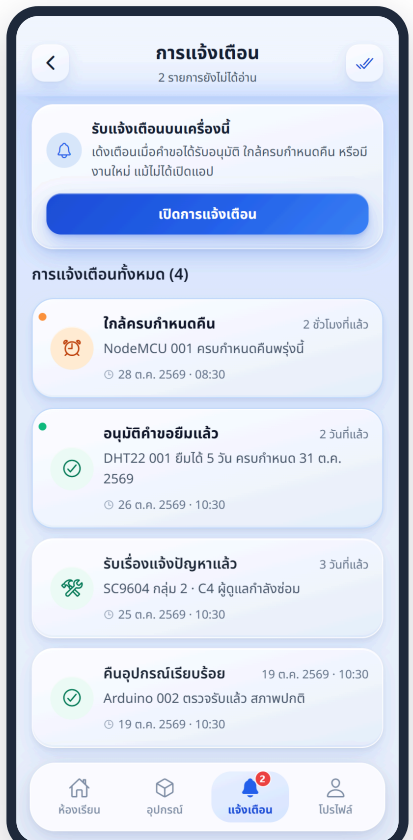
ประวัติ · โปรไฟล์ · แจ้งเตือน



ประวัติเรียงตามวัน แบบแอปธนาคาร



ยืมอยู่กี่ชิ้น เหลือสิทธิ์เท่าไร



ผลคำขอ + เตือนทุกวัน 08:00



แดชบอร์ด



กล่องคำขอ

อนุมัติได้ในไม่กี่วินาที

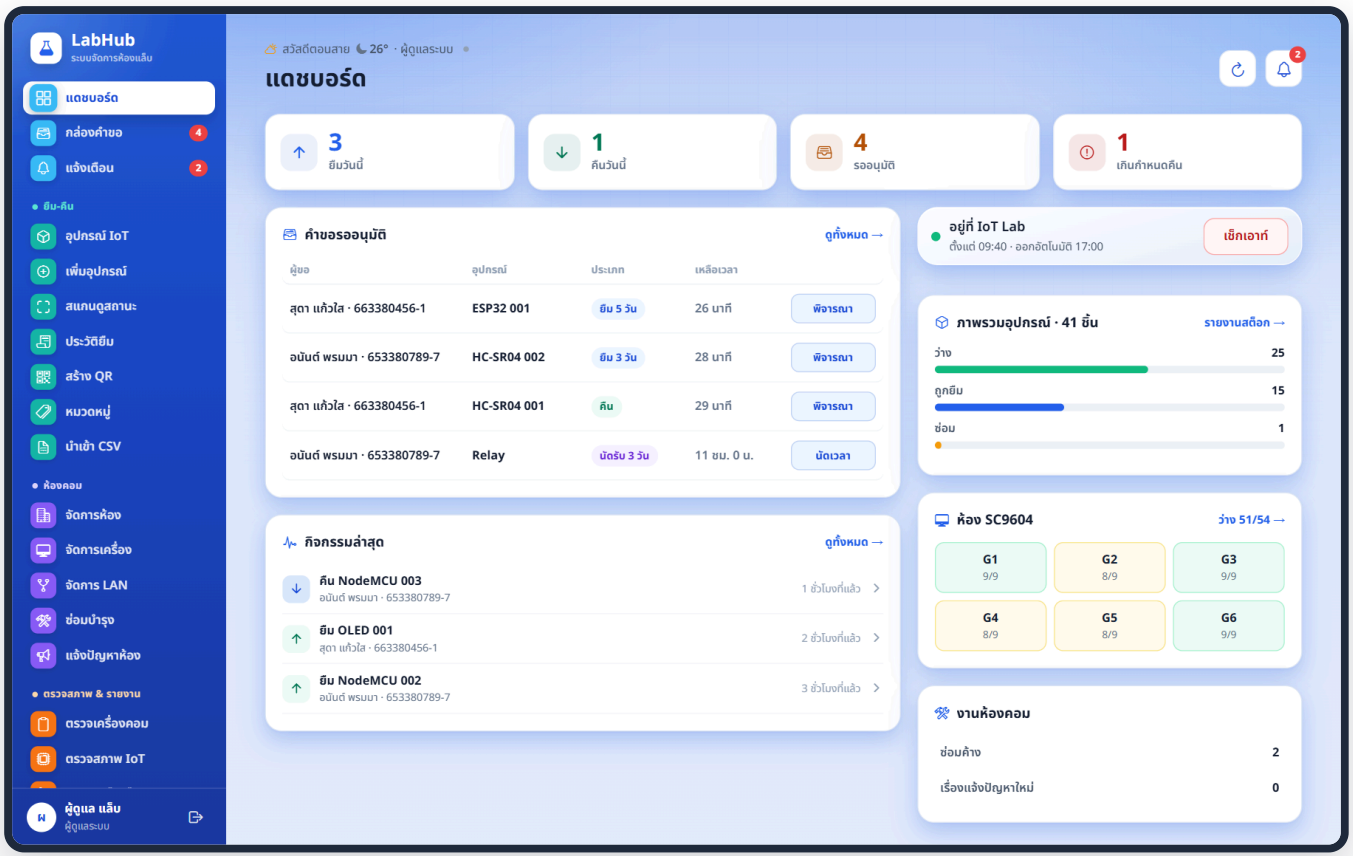
- มาถึงห้องกด**เช็กอิน** นักศึกษาเห็นว่ามีคนอยู่ (17:00 ระบบเช็กเอาท์ให้)
- คำขอใหม่**เด้งขึ้นเอง** ไม่ต้องรีเฟรช
- เห็นชื่อ รหัสนักศึกษา และ**รูปถ่าย** แล้วกดอนุมัติ
- ตอนคืนเทียบรูปก่อน-หลัง ชำรุดใส่ค่าเสียหายได้
- แท็บนัดรับ กดเลือกเวลานัดได้เลย

ลองพูดแบบนี้

นักศึกษากดส่งปุ๊บ คำขอเด้งที่เครื่องผู้ดูแลปั๊บ

● ฝั่งผู้ดูแล · บนคอม

เปิดบนคอม หน้าตาเปลี่ยนเป็นแบบจอใหญ่



เมนูซ้าย ตาราง หลายคอลัมน์ · ทำได้ทุกอย่างเหมือนมือถือ แค่จัดหน้าให้เหมาะกับจอใหญ่

ฝั่งผู้ดูแล · เครื่องมือ

เครื่องมืออื่น ๆ ของผู้ดูแล

เพิ่มอุปกรณ์

ถ่ายรูป ใส่ชื่อ ระบบ**ตั้งรหัสให้เอง** · นำเข้าจาก Excel ทีละหลายสิบชิ้นได้

พิมพ์ป้าย QR

เป็นแผ่นสติกเกอร์ A4 ไว้ติดของ

สแกนดูสถานะ

ใครยืมอยู่ คืนเมื่อไร ประวัติย้อนหลัง

รายงาน Excel / PDF

เลือกช่วงเวลาได้ ดูของและคนที่ยืมบ่อย

สต็อก + ตรวจประจำเทอม

เตือนประกันใกล้หมด ของเก่าที่ควรเปลี่ยน

จัดการคน + ตั้งค่า

ตั้ง TA/Admin · ปรับตัวเลขกติกาการยืม

ไอคอนฟ้า = TA ใช้ได้ · ไอคอนส้ม = Admin เท่านั้น



ผังห้องคอม รู้ก่อนเข้าห้อง

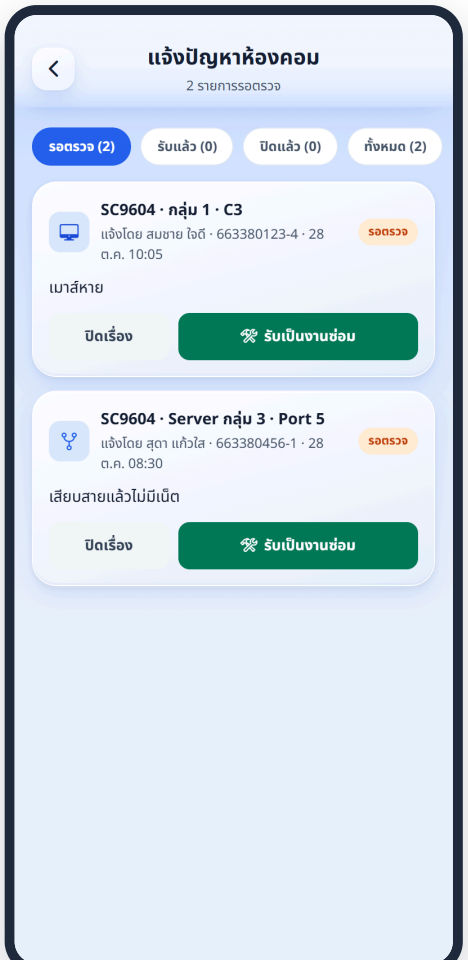
1 ห้อง = 6 กลุ่ม × คอม 9 เครื่อง + ตู้ Server กลุ่มละ 12 ช่อง LAN

- **ขาว+เขียว** ใช้ได้ · **เหลือง** ซ่อม · **แดง** เสีย
- จุดส้ม = เมาส์/คีย์บอร์ด/จอ มีปัญหา
- กด "เฉพาะที่ว่าง" ซ่อนเครื่องที่ใช้ไม่ได้
- แตะเครื่อง → **แจ้งปัญหาเครื่องนี้**

แจ้งปัญหาแล้วเกิดอะไรขึ้น

นศ. แจ้ง › เข้าคิว › ซ่อม › ใช้ได้

- ผู้ดูแลกด**รับเป็นงานซ่อม** หรือ**ปิดเรื่อง**พร้อมเหตุผล คนแจ้งได้แจ้งเตือน
- **กันแจ้งมั่ว:** เรื่องเดิมแจ้งซ้ำไม่ได้ · ไม่เกิน 5 ครั้ง/วัน
- มีเช็กลิสต์เมาส์/คีย์บอร์ด/จอ · ตรวจประจำเทอม · สถานะสาย LAN
- เครื่องเปลี่ยนสถานะเมื่อไร ใครเปลี่ยน **จดประวัติให้เอง**



● ใช้ได้บนอะไร



ใช้ได้ทั้งแอปและเว็บ

- แอปมือถือ iPhone และ Android
- เว็บ **labhub-kku.vercel.app**
- เว็บ**ติดตั้งเป็นไอคอน**บนมือถือได้ (PWA)
- **แจ้งเตือนเด้ง**ได้ แม้ไม่ได้เปิดแอป

ข้อจำกัด: ยืม/คืนต้องทำบน**มือถือ** (ต้องถ่ายรูปสด) · เข้าได้เฉพาะบัญชี มข. · ไม่มีหน้าสมัคร/ลืมรหัส

สร้างแอปมายังไง

แอปเหมือนร้านอาหาร

หน้าร้าน = ตัวแอป

React Native + Expo เขียนครั้งเดียว ได้ทั้ง iPhone, Android และเว็บ

แม่พิมพ์เดียว ปั๊มขนมได้ 3 แบบ

ครัว = หลังบ้าน

Supabase ฐานข้อมูล ล็อกอิน เก็บรูป อัปเดตสด งานตั้งเวลา ครบในที่เดียว

ครัวที่มีตู้เย็น เตา พนักงานพร้อม

บริการเสริม

Google ล็อกอิน มข. · **Vercel** ฝากเว็บ · **Web Push** แจ้งเตือน · **GitHub** เก็บโค้ด

เช่าที่ตั้งร้าน + คนส่งของ

ลองพูดแบบนี้

แอปแบ่งเป็น**หน้าร้าน**ที่ผู้ใช้เห็น กับ**ครัว**ที่เก็บข้อมูลและตัดสินกติกา หน้าร้านเขียนด้วย React Native ส่วนครัวใช้ Supabase

สร้างแอปมายังไง

ทำไมโกงไม่ได้ และปลอดภัย

กติกาอยู่ในครัว

แอปแค่ส่งคำขอ หลังบ้านเป็นคนตัดสิน แก้แอปก็โกงไม่ได้

ลูกค้ากรอกใบถอน พนักงานธนาคารเช็กยอด

เช็กสิทธิ์ทุกครั้ง

นักศึกษาเห็นแค่ข้อมูลของตัวเอง ไม่ล็อกอินเข้าไม่ได้

มี รปภ. เฝ้าทุกลิ้นชัก

30+
หน้าจอ

23
ตารางในฐานข้อมูล

53
ข้อทดสอบอัตโนมัติ

0 บาท
ใช้แพ็กเกจฟรี

เส้นทางการพัฒนา (ปี 2569)

มีนาคม	เริ่มโปรเจกต์ · ลอง MongoDB แล้ว**เปลี่ยนมาใช้ Supabase** เพราะมีครบในที่เดียว
พฤษภาคม	ระบบสแกน QR · ออกแบบหน้าตาใหม่ทั้งแอป (โทนฟ้า)
ต้นตุลาคม	ยืม-คืนเฟสแรกเสร็จ · บทบาท TA · ระบบห้องคอมครบ
10–11 ต.ค.	ฟีเจอร์ที่**อาจารย์แนะนำ**: ป๊อปอัปกฎ · ประวัติแบบแอปธนาคาร · เช็กอินผู้ดูแล · นัดรับ
12–19 ต.ค.	ล็อกอิน Google มข. · เว็บติดตั้งเป็นแอป + แจ้งเตือนเด้ง · หน้าจอใหญ่ผู้ดูแล · ชุดทดสอบ
22–28 ต.ค.	รองรับบุคลากร มข. · เสริมความปลอดภัย · จัดระเบียบโค้ด · แผนภาพรายงาน

ทริกโค้ด · เผื่ออาจารย์เปิดโค้ดให้ดู

โค้ดอยู่ตรงไหน + เปิดยังไง

app/	**หน้าจอ** 1 ไฟล์ = 1 หน้า · **app/scan.tsx** = หน้าสแกน
components/	ชิ้นส่วนหน้าจอที่ใช้ซ้ำ
lib/	ตัวช่วย เช่น ชื่อ/สีสถานะ สิทธิ์
supabase/migrations/	ฐานข้อมูล + กติกา (SQL)

- **GitHub:** github.com/Sineeyagron/Project1 → iot-lab-management → app → scan.tsx (หรือกด **t** แล้วพิมพ์ scan.tsx)
- **VS Code: Ctrl+P** พิมพ์ scan.tsx → **Ctrl+G** พิมพ์เลขบรรทัด
- **ทริก:** ค้นคำที่เห็นบนจอ เช่น **"ขอยืมอุปกรณ์นี้"** (VS Code **Ctrl+Shift+F**) เจอโค้ดของปุ่มนั้นทันที
- เลขบรรทัดไม่ตรง? ค้นชื่อแทน เช่น **atLimit**, **formReady**

เปิดไฟล์ **app/scan.tsx** ค้างไว้ในแท็บก่อนพรีเซนต์ · เดี๋ยวจะมีรูปโค้ดจริงจากไฟล์นี้ 3 รูป **ไม่มีคอมเมนต์** เลขบรรทัดตรงกับไฟล์ (29 ต.ค.)

● ทริกโค้ด · ปูพื้น syntax

หน้าตาโค้ด 12 แบบ แปลเป็นไทย

`const ก = …`	ตั้งชื่อ ก ให้ค่านี้ (ตัวแปร) · แปะป้ายบนกล่อง
`"ข้อความ"`	ข้อความตรง ๆ
`` `…${ค่า}…` ``	ข้อความที่เติมค่าลงไป · แบบฟอร์มเว้นช่อง
`ก.ข`	**ข ของ ก** · item.id = รหัสของชิ้นนี้
`{ ช่อง: ค่า }`	กล่องข้อมูลหลายช่อง · ใบฟอร์ม
`ก[ข]`	เปิด ก ที่ป้าย ข · เปิดพจนานุกรม

`ชื่อ(…)`	**สั่งให้ทำงาน** ในวงเล็บ = ของที่ยื่นให้
`() => …`	เมื่อถูกเรียก ให้ทำ… · โน้ต "ถ้ากด ให้ทำ"
`if (…)`	ถ้า … ให้ทำ
`await`	รอให้เสร็จก่อน · รอโอนเงินผ่าน
`try / catch`	ลองทำ พังไปทาง catch · throw = โยนปัญหา
`<ชื่อ ค่า={…}>`	**ชิ้นส่วนบนจอ** + ตั้งค่าให้มัน · เลโก้

โค้ด 1 บรรทัด ≈ **1 ประโยคคำสั่ง** อ่านซ้ายไปขวา · **;** = จบประโยค · ทุกแบบในหน้านี้จะเจอในรูปโค้ด 3 รูปข้างหน้า

ทริกโค้ด · method คืออะไร

Method = คำสั่งสำเร็จรูปที่มีชื่อ

เรียกชื่อ + ( ) แล้วมันทำงานให้ เหมือนกดปุ่ม "อุ่นข้าว" ที่ไมโครเวฟ · **มีจุดข้างหน้า** (supabase.rpc) = มาจากเครื่องมือภายนอก · **ไม่มีจุด** = ทีมเขียนเอง

`supabase.rpc("ชื่อ", {…})`	**โทรหาหลังบ้าน** ขอบริการตามชื่อ แนบฟอร์มไป	บรรทัด 194
`uploadBorrowPhoto(รูป)`	ย่อรูปแล้ว**อัปโหลด** ได้ที่อยู่รูปกลับมา	บรรทัด 192
`openForm("borrow")`	**เปิดฟอร์มยืม** ล้างค่าเก่า ตั้งวันเริ่มต้น	บรรทัด 380
`runLookup(รหัส)`	**ค้นชิ้นนี้ใหม่** ถามหลังบ้านว่าสถานะอะไร	บรรทัด 207
`notify("หัวข้อ", "…")`	**เด้งป๊อปอัป**ที่มีปุ่มโอเค	บรรทัด 210
`setXxx(…)`	ขึ้นต้นด้วย **set** = เปลี่ยนค่าที่จอจำไว้ แล้วจอวาดใหม่เอง	บรรทัด 203–204
`ข้อความ.trim()`	**ตัดช่องว่าง**หัวท้าย (เว้นวรรคอย่างเดียว = ยังไม่ได้เขียน)	บรรทัด 175

สรุป: อ่านโค้ดไม่มีคอมเมนต์ยังไง

- **1. หาภาษาไทยในโค้ด** = คำที่ขึ้นบนจอ รู้เลยว่าตรงนี้คือส่วนไหนของหน้า
- **2. อ่านชื่อเป็นภาษาอังกฤษ** atLimit = ถึงเพดาน · formReady = ฟอร์มพร้อม · photoUri = รูป · condition = สภาพ · ขึ้นต้นด้วย set = เปลี่ยนค่าบนจอ
- **3. แปลสัญลักษณ์** ตามตารางข้าง ๆ

`===`	เท่ากับ · **!==** ไม่เท่ากับ
`>=`	มากกว่าหรือเท่ากับ
`&&`	และ (ต้องจริงทุกข้อ)
`||`	หรือ (จริงข้อเดียวก็พอ)
`! / !!`	ไม่ / มีค่าแล้วไหม
`A ? B : C`	ถ้า A ใช้ B ไม่งั้นใช้ C
`??`	ถ้าไม่มีค่า ใช้ค่านี้แทน
`await`	รอให้เสร็จก่อน

● ทริกโค้ด · ลองแยกทีละคำ

หั่นเป็นชิ้น → แปล → อ่านเป็นประโยคไทย

บรรทัด 363

```
const atLimit = activeCount !== null && activeCount >= max;
```

`const atLimit =`	ตั้งชื่อ "ถึงเพดาน" =
`!== null`	ไม่ใช่ค่าว่าง (รู้ตัวเลขแล้ว)
`&&`	และ
`>= max`	มากกว่าหรือเท่ากับ เพดาน

อ่านรวม: "ถึงเพดาน คือ รู้จำนวนที่ยืมแล้ว และจำนวนนั้นไม่น้อยกว่าเพดาน"

บรรทัด 192

```
const photoPath = await uploadBorrowPhoto(photoUri);
```

`const photoPath =`	ตั้งชื่อ "ที่อยู่รูป" =
`await`	รอให้เสร็จ
`uploadBorrowPhoto`	สั่งอัปโหลดรูปหลักฐาน
`(photoUri)`	ยื่นรูปที่เพิ่งถ่ายให้

อ่านรวม: "อัปโหลดรูปที่ถ่าย รอจนเสร็จ แล้วจดที่อยู่รูปไว้"

● ทริกโค้ด · รูปที่ 1 · app/scan.tsx

ปุ่ม "ขอยืมอุปกรณ์นี้" กดไม่ได้เมื่อยืมครบ

app/scan.tsx · บรรทัด 362–363, 378–386 · … = ตัดท้ายบรรทัดให้พอดีจอ

```
362 const max = lookup.max_active_borrows ?? 3;
363 const atLimit = activeCount !== null && activeCount >= max;
    ⋮
378 <TouchableOpacity
379   style={[s.primaryBtn, atLimit && { opacity: 0.45 }]}
380   onPress={() => openForm("borrow")}
381   disabled={atLimit}
382   activeOpacity={0.85}
383 >
384   <Ionicons name="hand-left-outline" ... />
385   <Text ...>ขอยืมอุปกรณ์นี้</Text>
386 </TouchableOpacity>
```

385	เจอคำไทย = **นี่คือปุ่มขอยืม**
378	TouchableOpacity = ของที่แตะได้ = ตัวปุ่ม
363	atLimit = ยืมอยู่ ≥ เพดาน (3)
381	ถึงเพดาน ⟶ **ปุ่มกดไม่ได้**
379	ถึงเพดาน ⟶ ปุ่มจาง 45%
380	กดแล้ว ⟶ เปิดฟอร์มยืม

พูดว่า: "บรรทัด 363 เช็กว่ายืมครบหรือยัง ถ้าครบ บรรทัด 381 สั่งให้ปุ่มกดไม่ได้ ส่วนหลังบ้านก็เช็กซ้ำอีกชั้น"

● ทริกโค้ด · รูปที่ 2 · app/scan.tsx

ปุ่ม "ส่งคำขอ" กดได้เมื่อไร

app/scan.tsx · บรรทัด 171–175

```
171 const needsEvidence = mode === "borrow" || mode === "return" || mode === "claim";
172 const formReady =
173   !!mode &&
174   (mode === "return" || !!days) &&
175   (!needsEvidence || (!!condition && !!photoUri && (condition === "good" || !!note.trim())));
```

171	ต้องมีหลักฐานไหม = ใช่ ถ้า**ยืม** หรือ**คืน** หรือ**รับตามนัด**
172	ฟอร์มพร้อมส่ง = ต้องผ่าน**ครบ 3 ข้อ** (&&)
173	ข้อ 1: เลือกแล้วว่าจะทำอะไร
174	ข้อ 2: เป็นการคืน หรือเลือกวันแล้ว
175	ข้อ 3: เลือกสภาพ **และ** ถ่ายรูป **และ** (ปกติ **หรือ** เขียนหมายเหตุ)
535	disabled={!formReady} = ยังไม่พร้อม ปุ่มส่งกดไม่ได้

พูดว่า: "บรรทัด 175 คือเหตุผลที่ยืมโดยไม่ถ่ายรูปไม่ได้ ปุ่มส่งกดได้ก็ต่อเมื่อเลือกสภาพ ถ่ายรูปแล้ว และถ้าชำรุดต้องเขียนว่าตรงไหน"

● ทริกโค้ด · รูปที่ 3 · app/scan.tsx

กดส่งแล้ว เกิดอะไรขึ้น

app/scan.tsx · บรรทัด 192–210 ในฟังก์ชัน submit · … = ตัดบางส่วนให้พอดีจอ (ฉบับเต็มมีครบ)

```
192 const photoPath = await uploadBorrowPhoto(photoUri);
193 ({ error } = mode === "borrow"
194   ? await supabase.rpc("request_borrow", {
195       p_item_id: item.id, p_days: days, ...
196     })
197   : await supabase.rpc("request_return", {
198       p_item_id: item.id, p_condition: condition, ...
199     }));
    ⋮
202 if (error) throw error;
203 setConfirmOpen(false);
204 setJustSent(mode === "claim"
205   ? `รับ ${item.item_code} ตามนัดแล้ว ...`
206   : `ส่งคำ${KIND_TH[mode]} ${item.item_code} แล้ว รอผู้ดูแลอนุมัติ`);
207 await runLookup(item.id);
208 } catch (e: any) {
210   notify("ส่งคำขอไม่สำเร็จ", e.message || ...);
```

192	**อัปโหลดรูป**ก่อน รอจนเสร็จ
193–199	ยืม (?) → request_borrow · ไม่งั้น (:) คืน → request_return
202	หลังบ้านตอบว่า**มีปัญหา** → ไปที่ catch
204–206	สำเร็จ → "ส่งคำขอยืม … แล้ว รอผู้ดูแลอนุมัติ"
207	สแกนชิ้นเดิมซ้ำ จอเปลี่ยนเป็นรออนุมัติ
208–210	พลาด → เด้ง "ส่งคำขอไม่สำเร็จ" + เหตุผล

พูดว่า: "แอปอัปโหลดรูป แล้วส่งคำขอไปหลังบ้าน หลังบ้านเป็นคนตัดสิน ผ่านก็รออนุมัติ ไม่ผ่านก็เด้งเหตุผลบอก"

วันพรีเซนต์

แบ่งกันพูด + ของที่ต้องเตรียม

แนะนำ + ภาพรวม	5 นาที	คนที่ 1
เดโมนักศึกษา	6 นาที	คนที่ 1
เดโมผู้ดูแล	5 นาที	คนที่ 2
ห้องคอม	4 นาที	คนที่ 2
สร้างยังไง	4 นาที	คนที่ 1
สรุป + ถามตอบ	1 นาที	ทั้งคู่

- **มือถือ** บัญชีนักศึกษาทดสอบ อนุญาตกล้องแล้ว
- **โน้ตบุ๊ก** บัญชี TA/Admin (ต้องคนละบัญชี)
- **ของทดสอบ 1 ชิ้น** ติด QR สถานะว่าง
- **เปิดแอปเล่นก่อน 1 วัน** (แพ็กเกจฟรีจะหลับถ้าไม่มีคนใช้ราว 7 วัน)
- ฮอตสปอตสำรอง + **วิดีโอเดโมสำรอง**

วันพรีเซนต์ · เดโมสด

ลำดับเดโม

1	คนที่ 1 โชว์หน้าแรก · คนที่ 2 กดเช็กอิน → แถบในมือถือเปลี่ยนเป็นสีเขียวเอง
2	คนที่ 1 แท็บอุปกรณ์ → สแกน QR → ขอยืม → ถ่ายรูป → ยอมรับกฎ → ส่ง
3	คนที่ 2 คำขอเด้งบนโน้ตบุ๊ก → ดูรูป → อนุมัติ
4	คนที่ 1 เปิดแจ้งเตือน → ประวัติ → สแกนชิ้นเดิม ขอคืน · คนที่ 2 รับคืน
5	คนที่ 2 ผังห้องคอม · คนที่ 1 แจ้งปัญหา "ทดสอบระบบ" · คนที่ 2 **ปิดเรื่อง**
6	คนที่ 2 (ถ้ามีเวลา) ออกรายงาน PDF

ของที่ทำตอนเดโมเข้าระบบจริง ใช้ของ/บัญชีทดสอบเท่านั้น · สแกนไม่ติด → พิมพ์รหัส 4 ตัว · คำขอไม่เด้ง → กดรีเฟรช · เน็ตล่ม → เปิดวิดีโอสำรอง

อ่านหน้านี้ก่อนขึ้นพูด

สรุป + คำถามยอดฮิต

- **LabHub** = ยืม-คืนอุปกรณ์ + ห้องคอม ในแอปเดียว
- **ยืม:** สแกน → ถ่ายรูป → ยอมรับกฎ → ผู้ดูแลอนุมัติ
- **กติกา:** 3 ชิ้น · 3/5/7 วัน · ยืมต่อ 1 ครั้ง · เตือน 08:00
- **สร้างด้วย:** React Native (หน้าร้าน) + Supabase (ครัว)

ยืมแล้วไม่คืน?

เตือนทุกวัน เลยกำหนดยืมต่อไม่ได้ ของค้างนับโควตา ยืมชิ้นใหม่ไม่ได้

ของคืนมาเสีย?

เทียบรูปก่อน-หลัง ใส่ค่าเสียหายได้

ไม่รู้คำตอบ?

"ขอไปเช็กแล้วแจ้งกลับนะครับ/ค่ะ" ไม่ผิด · คำถามอื่นอยู่ในฉบับเต็ม

ประโยคปิด: "LabHub ทำให้การยืม-คืนของและการดูแลห้องคอม มีหลักฐาน ตรวจสอบได้ และเลิกใช้กระดาษ"